

JVMud User Manual

JVMud Project

Version 0.1.0-SNAPSHOT

Table of Contents

Status	1
How To Read This Manual	2
Terminology	3
Getting Oriented	4
Overview	4
The JVMud Philosophy Of MUD	4
Game	4
Text	4
Interactive	4
Multiplayer	4
World	5
Persistence	5
Temporality	5
Presence	5
LPC Compatibility And Engine Vocabulary	5
Repository Tour	6
Installing And Running	7
Installing	7
Running JVMud	7
Connecting With Telnet	7
Smoke Tests	7
Playing And Testing A World	8
Login Flow	8
Basic Commands	8
Command Routing	8
Transcript Regressions	8
Operating JVMud	9
Admin CLI	9
Runtime Model	9
Mudlib Boundary	9
LPC And Mudlib Authoring	10
Supported LPC	10
Explicit Types	10
Efuncs	10
Lifecycle Hooks	10
Compatibility Shims	10
Mudlibs	11
Layout	11

LP245 Notes	11
Upstream Source Policy	11
Developer Reference	12
Compiler Pipeline.	12
Runtime And Server Internals	12
Testing And Verification.	12
Troubleshooting	13
Build Failures	13
Object Load Failures	13
Login And Command Failures	13
Documentation Generation.	13
Appendixes	14
Commands.	14
Configuration Reference	14
Compatibility Notes.	14
Glossary	14

Status

Content to be written.

How To Read This Manual

Content to be written.

Terminology

Content to be written.

Getting Oriented

Content to be written.

Overview

Content to be written.

The JVMud Philosophy Of MUD

JVMud is built around eight core concepts:

- Game
- Text
- Interactive
- Multiplayer
- World
- Persistence
- Temporality
- Presence

Together, these concepts define what JVMud means by MUD.

JVMud is not a general-purpose simulation framework, chat server, collaboration platform, generic game engine, or generic MUD framework. It is specifically and by design an LPMud engine: a platform for imagining, creating, operating, and playing LPC-authored game worlds in the tradition of LP MUDs.

Game

The world exists to be played. JVMud assumes rules, mechanics, goals, challenges, progression, or other game-like structures.

Text

Text is the primary medium through which the world is perceived and manipulated. It is a first-class part of the engine's design, not a skin over a visual game.

Interactive

Players can affect the world, and the world can affect players. Actions have consequences.

Multiplayer

Multiple players inhabit the same world simultaneously. The world is shared.

World

The game takes place within a concrete virtual domain. JVMud describes this domain in terms of Places, Links, Entities, and Movement.

Places give the world locality. Links make Places traversable. Entities are the things that exist in that structure. Movement changes where an Entity is present.

Persistence

The world and its state endure independently of any individual player's session. A player may disconnect and later return to a world whose state has continued to exist.

Persistence does not require eternal continuity. Reboots, resets, and selected save/restore behavior can all be compatible with a persistent world.

Temporality

The world can change through time. Events occur, actions have consequences, and state evolves.

Temporality is separate from Persistence: a world can have active time, saved state, restored state, and reset state without treating all of those as the same idea.

Presence

Presence means that a player experiences the game from within the World. The player is not an outside observer manipulating a simulation from above.

In JVMud terms, a player participates through a Persona: an Entity that serves as the player's current point of perception and action. A Persona is located somewhere, and that location determines what the player can perceive and what actions are locally available.

Players are somewhere, not everywhere. Presence creates locality of information, which makes exploration, discovery, concealment, surprise, navigation, and mystery possible.

LPC Compatibility And Engine Vocabulary

JVMud has one mudlib and language target: LPC for LPMud-style worlds. Its compatibility work exists to make JVMud a deeper LPMud platform, not to broaden it into a multi-language MUD framework.

An LPMud, in JVMud's sense, is a game world authored in LPC, compiled into live game objects, and able to be rewritten, recompiled, and reloaded while the game continues running.

That does not mean JVMud adopts every legacy LPMud driver term as engine ontology.

Legacy names such as rooms, objects, heartbeats, applies, call_outs, and master objects may appear in compatibility surfaces, mudlib source, and LPC-facing documentation. The engine-facing model remains JVMud's own model: Game, Text, Interactive, Multiplayer, World, Persistence, Temporality, and Presence, built from Places, Links, Entities, Players, Sessions, and Personas.

Repository Tour

Content to be written.

Installing And Running

Content to be written.

Installing

Content to be written.

Running JVMud

Content to be written.

Connecting With Telnet

Content to be written.

Smoke Tests

Content to be written.

Playing And Testing A World

Content to be written.

Login Flow

Content to be written.

Basic Commands

Content to be written.

Command Routing

Content to be written.

Transcript Regressions

Content to be written.

Operating JVMud

Content to be written.

Admin CLI

Content to be written.

Runtime Model

Content to be written.

Mudlib Boundary

Content to be written.

LPC And Mudlib Authoring

Content to be written.

Supported LPC

Content to be written.

Explicit Types

Content to be written.

Efuncs

Content to be written.

Lifecycle Hooks

Content to be written.

Compatibility Shims

Content to be written.

Mudlibs

Content to be written.

Layout

Content to be written.

LP245 Notes

Content to be written.

Upstream Source Policy

Content to be written.

Developer Reference

Content to be written.

Compiler Pipeline

Content to be written.

Runtime And Server Internals

Content to be written.

Testing And Verification

Content to be written.

Troubleshooting

Content to be written.

Build Failures

Content to be written.

Object Load Failures

Content to be written.

Login And Command Failures

Content to be written.

Documentation Generation

Content to be written.

Appendixes

Content to be written.

Commands

Content to be written.

Configuration Reference

Content to be written.

Compatibility Notes

Content to be written.

Glossary

Content to be written.